



K210 Vision Module Tutorial

Perface

Our Company

ACEBOTT STEM Education Tech Co.,Ltd

Founded in China's Silicon Valley in 2013, ACEBOTT is a STEM education solution leader. We have a team of 150 individuals, including members from research and development, sales, and logistics. Our goal is to provide high-quality STEM education products and services to our customers. We are working together with STEM education experts and our business partners to produce successful STE products together. Our self-owned factory also provides OEM services for our clients, including logo customization on product packaging and PCB.

Tutorial

This is a hands-on course designed for beginners, aiming to guide students into the world of programming, electronics, and robotics based on the K210 vision module. In this course, students will learn the control theory knowledge and practical application of the K210 vision module and use a smart car with a K210 vision module.

With this kit, you can:

1. Understand the structure and function of the K210 vision module.
2. Understand the concepts and applications of machine vision.
3. Master the image processing methods of K210 vision module.
4. Master the principles and functions of color recognition, face recognition, road sign recognition, and intelligent line patrol of the K210 vision module.
5. Improve your maker skills by following the step-by-step tutorial to complete the K210 Vision Module Car project using the ACEBOTT kit.

In general, the ACEBOTT K210 Vision Module Learning Kit is an expansion kit designed specifically for beginners. Using this kit, you can master the basic working principles and usage of the K210 vision module, and have the ability to apply the knowledge you have learned to solve practical problems.

Customer service

ACEBOTT is a dynamic and fast-growing STEM education technology company that strives to offer excellent products and quality services that meet your expectations. We value your feedback and encourage you to drop us a line at **support@acebott.com** with any comments.

or suggestions you may have.

Our experienced engineers are dedicated to promptly addressing any problems or questions you may have about our products. We guarantee a response within 24 hours during business days.

Follow Us

Scan the QR codes to Follow Us for troubleshooting & the latest news.

We have a very large community that is very helpful for troubleshooting and we also have a support team at the ready to answer any questions.



ACEBOTT FB QR Code



YouTube QR Code

Contents

Lesson 1 Introduction to Machine Vision	1
I . The Concept of Machine Vision	1
II . Application of Machine Vision	1
III. Visual Module Introduction	2
Lesson 2 Color Recognition	5
I . The Principle of Color Recognition	5
II . Application Scenarios of Color Recognition	6
III. K210 Module Recognition Color	7
Lesson 3 Color Tracing	10
I . Identify Fixed Colors	10
II . Color Tracking Principle	11
III. Color Tracking Program	13
Lesson 4 Face Recognition	16
I . The Principle of Face Recognition	16
II . Face Detection	18
III. Facial Recognition Starts the Smart Car	21
Lesson 5 Road Sign Recognition	23
I . Principles of Road Sign Identification	23
II . Application of Road Sign Recognition	24
III. Procedure for Road Sign Identification	25
IV. Line Inspection and Road Sign Recognition	27
Lesson 6 Visual Line Patrol	30
I . The Principle of Visual Line Patrol	30
II . Visual Line Patrol Algorithm	30
III. Intelligent Car Visual Line Patrol Principle	32
IV. Intelligent Car Visual Line Patrol Program	34
Lesson 7 Machine Learning	38
I . Machine Learning	38
II . Image Classification	39
III. Built-in Image Classification Application	40
IV. Machine learning Image Classification	41
Lesson 8 APP Control	43
I . APP Download	43
II . Program upload	44
III. APP control instructions	45

Lesson 1 Introduction to Machine Vision

I . The Concept of Machine Vision

Machine Vision is a technology system that simulates human visual functions through optical devices (such as cameras, lenses) and sensors, aiming to give machines the ability to "perceive-analyze-decision" and replace human eyes to complete detection, measurement, identification and control tasks. Its essence is to convert the target object into a digital image signal, use algorithms to extract feature information, and ultimately drive the device to perform actions. It is widely used in industrial automation, intelligent manufacturing, autonomous driving, security monitoring and other fields.



II . Application of Machine Vision

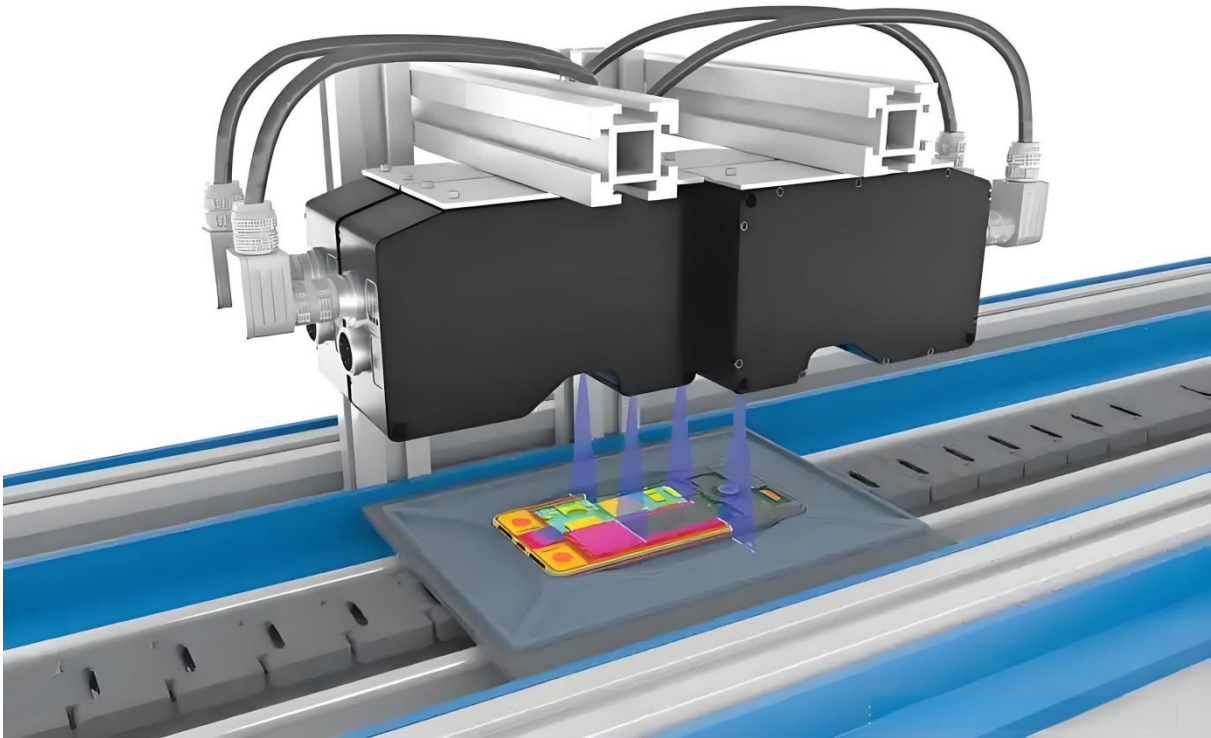
1. **Smart Agriculture:** used for crop disease and pest identification,

growth status monitoring, etc., to help farmers improve crop yields and quality.

2. Smart Home: used for face recognition, gesture recognition, etc. to realize intelligent control and personalized services of smart home.

3. Smart Park: used for personnel access management, vehicle identification, etc. to improve the security and management efficiency of the park.

4. Industrial Internet of Things: used for product quality inspection, production line automation control, etc. to improve the efficiency and quality of industrial production.



III. Visual Module Introduction

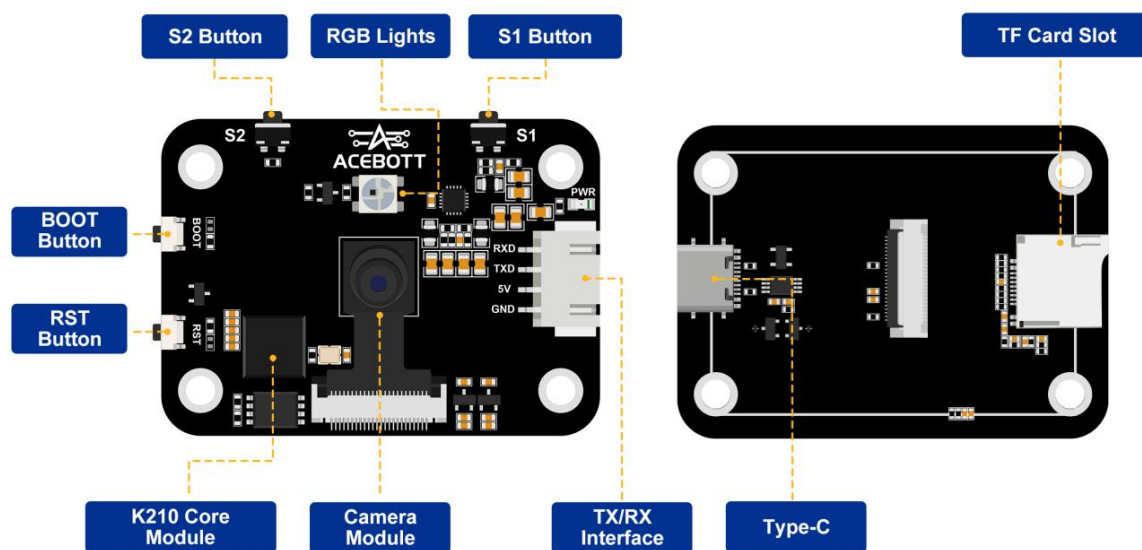
The K210 vision module is a powerful AI vision module that includes a neural network hardware accelerator KPU, which can perform high-performance convolutional neural network operations and high-performance machine vision and auditory processing. It can support 15 visual functions including face recognition, hand recognition, mask

recognition, image code recognition, feature detection, digital recognition, multi-color recognition, license plate recognition, road sign recognition and visual line patrol.



Specifications

- **Button S1/S2:** You can customize the button function.
- **RGB light:** It can be programmed to display red, green, blue, white and other colors.
- **TF card slot:** It used to insert TF card, with the gold finger facing the module, to save programs, models and other files.
- **Fixed copper pillars:** Four fixed copper pillars in total, M3 size.
- **K210 core module:** It contains the minimum operating system of the K210 chip.
- **Camera:** Collects images.
- **RST button:** Reset button, used to restart the K210 chip.
- **BOOT button:** Load the startup program.
- **TX/RX interface:** Connected to the controller board to transmit data.
- **Type-C interface:** Download firmware, serial port debugging, connect to IDE, etc.
- **CH340 chip:** Connected to the Type-C interface to convert USB signals into TTL signals.

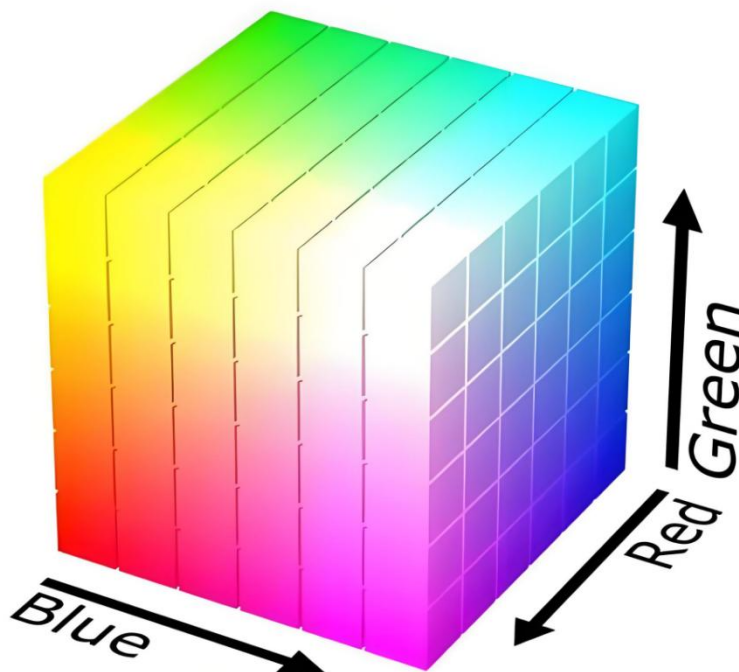


Lesson 2 Color Recognition

I . The Principle of Color Recognition

The core principle of machine color recognition is optical sensor technology. An optical sensor is a device that obtains the characteristics of an object by measuring the intensity and frequency of a light signal. In robotic applications, optical sensors usually use three photodiodes (LEDs) of red, green, and blue to emit light beams of different colors, and identify the color of an object by measuring the intensity of the reflected light signal returned.

The sensor converts the red, green and blue signals into digital signals and transmits them between computers. These digital signals are converted into a two-dimensional array by the processing program to form a color image. The processing program combines the colors of adjacent pixels and identifies the color of the object by comparing the ratio of different colors.



Steps for color recognition:

Light collection: The camera or optical sensor captures the light reflected or transmitted by the target object. These lights contain information of different wavelengths, corresponding to different colors.

Photoelectric conversion: The sensor converts the received light signal into an electrical signal. This process is usually completed by CCD (charge coupled device) or CMOS (complementary metal oxide semiconductor).

Color decomposition: The composite light is decomposed into three basic colors (red, green and blue) (RGB) through a color filter or a spectroscope. The color information of each pixel is recorded separately.

Digital conversion: Analog electrical signals are further converted into digital signals, forming a data format that can be processed by computers.

Image processing and analysis: Use image processing algorithms to analyze the acquired data, determine the color value of each pixel, and judge the overall color category according to certain rules. This may include color space conversion (such as from RGB to HSV), threshold segmentation, cluster analysis and other techniques.

Result output: Finally, the system will give the recognized color name or the corresponding numerical representation.

II . Application Scenarios of Color Recognition

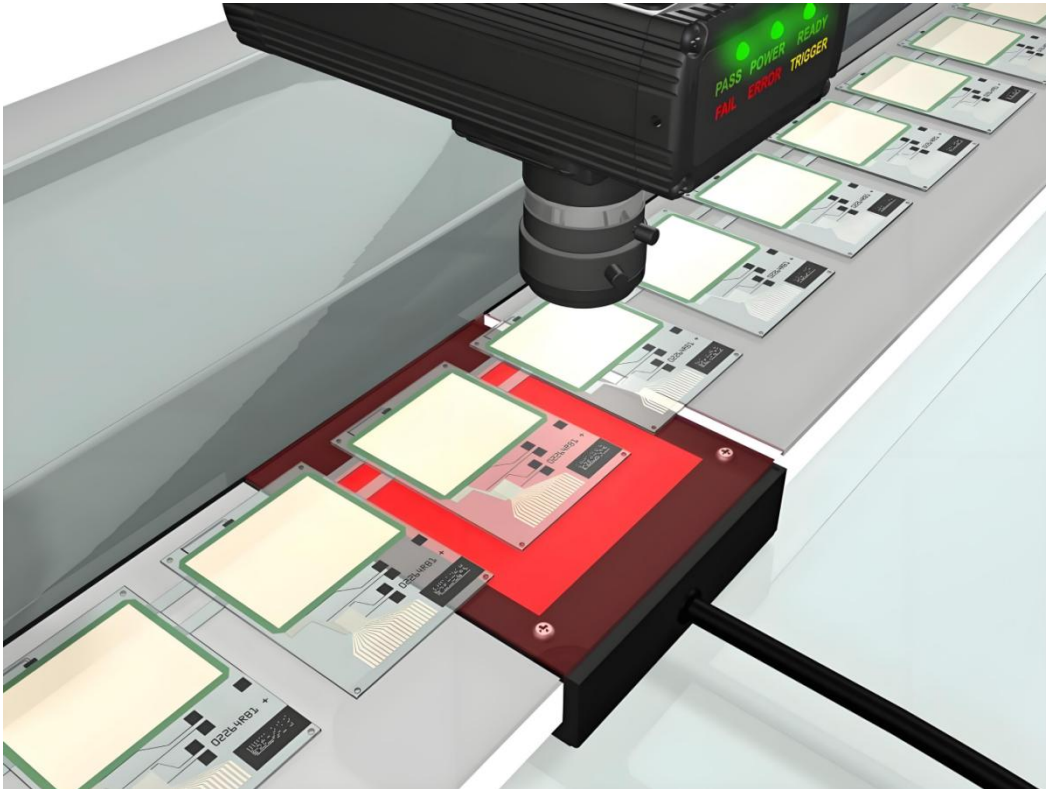
Color recognition technology is widely used in many fields:

- **Industrial Automation:** Detect product color on the production line to ensure color consistency.
- **Robotics:** Used to recognize the color of objects and perform operations such as sorting, grabbing, and placing.

■ **Smart Home:** Identify the color of indoor light and automatically adjust the brightness and color temperature of lighting equipment.

■ **Consumer Electronics:** Identify screen colors in smartphones, tablets, and other devices to enable color calibration and adjustment.

Through these principles and applications, color recognition technology can be effectively applied in various scenarios to improve the level of automation and intelligence.



III. K210 Module Recognition Color

1. Sample Program

Open the ["2.Color_recognition.py"](#) file located in "English\Python\3.Program". Connect the ESP32 controller board to the computer using a USB cable, select the correct controller board and port, upload the code to the ESP32 controller board, and then connect the K210 module to the ESP32 controller board.

Note: Before uploading the program, please confirm whether the library files of the K210 module have been added. If not, please open "Read Before Use" and follow the prompts to complete the addition.

The reference procedure is as follows:

```
from libs import ACB_Canmv
import time

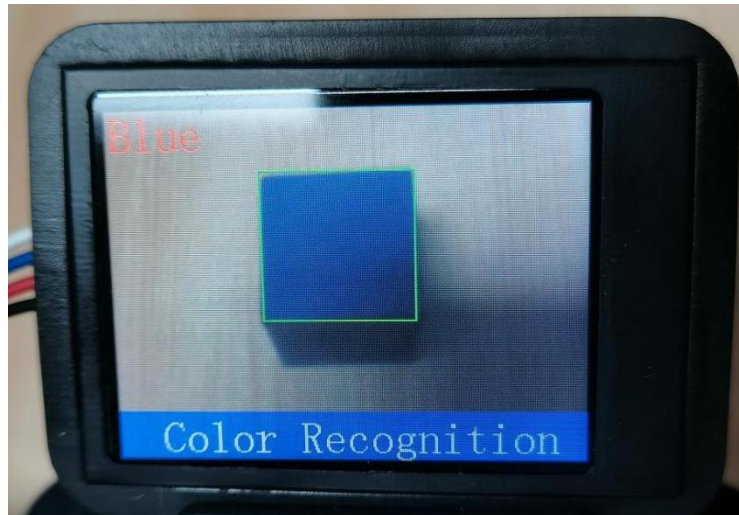
cam = ACB_Canmv.ACB_Canmv()
COLOR_INDEX = 3
PIXELS_TH = 200
AREA_TH = 200
cam.init(cam.SDA, cam.SCL)

while True:
    if cam.color_recognize(COLOR_INDEX, PIXELS_TH, AREA_TH):
        print("Tag:", cam.getTag())
        print("X,Y,W,H:", cam.getX, cam.getY, cam.getW, cam.getH)
        time.sleep_ms(50)
```

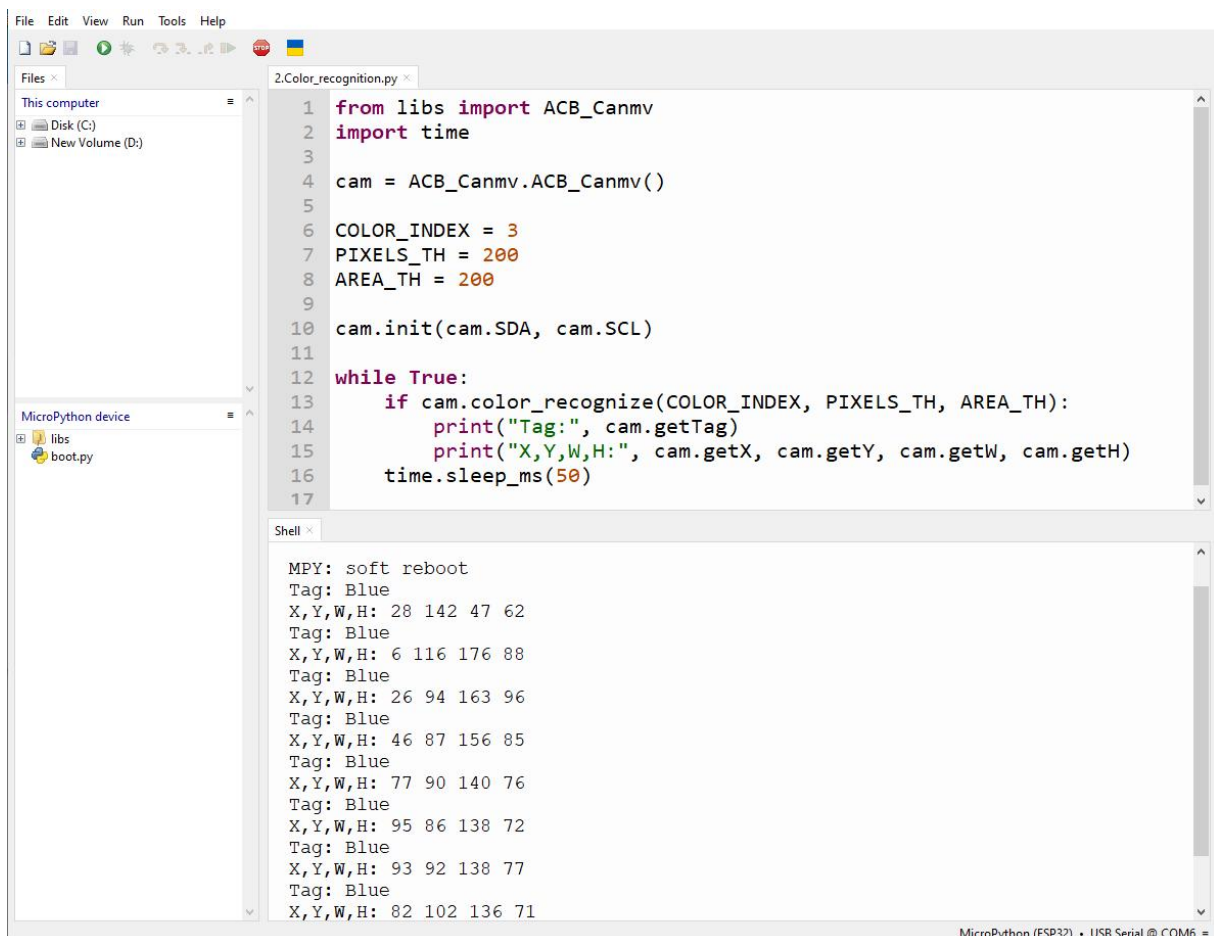
2. Program Functions

When the program is uploaded, place the cards or objects with red, green and blue colors in front of the camera of the K210 module in turn. The screen will use a box to select the color of the card or object and display its color label.

Note: It currently only recognizes three colors: red, green and blue.



Open the Thonny console, and the color label of the object block, position information, and the values of the length and height of the box will be printed.



```
File Edit View Run Tools Help
2.Color_recognition.py
1 from libs import ACB_Canmv
2 import time
3
4 cam = ACB_Canmv.ACB_Canmv()
5
6 COLOR_INDEX = 3
7 PIXELS_TH = 200
8 AREA_TH = 200
9
10 cam.init(cam.SDA, cam.SCL)
11
12 while True:
13     if cam.color_recognize(COLOR_INDEX, PIXELS_TH, AREA_TH):
14         print("Tag:", cam.getTag())
15         print("X,Y,W,H:", cam.getX(), cam.getY(), cam.getWidth(), cam.getHeight())
16         time.sleep_ms(50)
17
Shell
MPY: soft reboot
Tag: Blue
X,Y,W,H: 28 142 47 62
Tag: Blue
X,Y,W,H: 6 116 176 88
Tag: Blue
X,Y,W,H: 26 94 163 96
Tag: Blue
X,Y,W,H: 46 87 156 85
Tag: Blue
X,Y,W,H: 77 90 140 76
Tag: Blue
X,Y,W,H: 95 86 138 72
Tag: Blue
X,Y,W,H: 93 92 138 77
Tag: Blue
X,Y,W,H: 82 102 136 71
MicroPython (ESP32) • USB Serial @ COM6
```

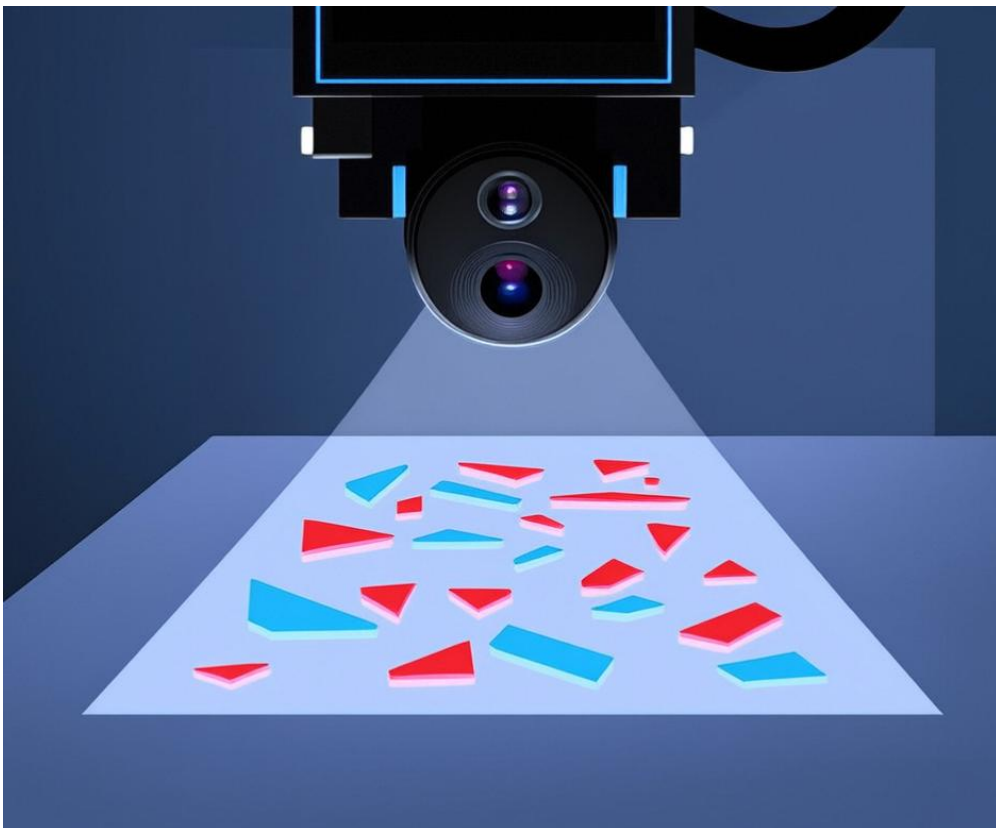
Lesson 3 Color Tracing

I . Identify Fixed Colors

Color tracking is an important application in the field of image processing, which refers to identifying areas of specific colors from an image and then performing specific tasks.

Color tracking is mainly carried out through the following steps:

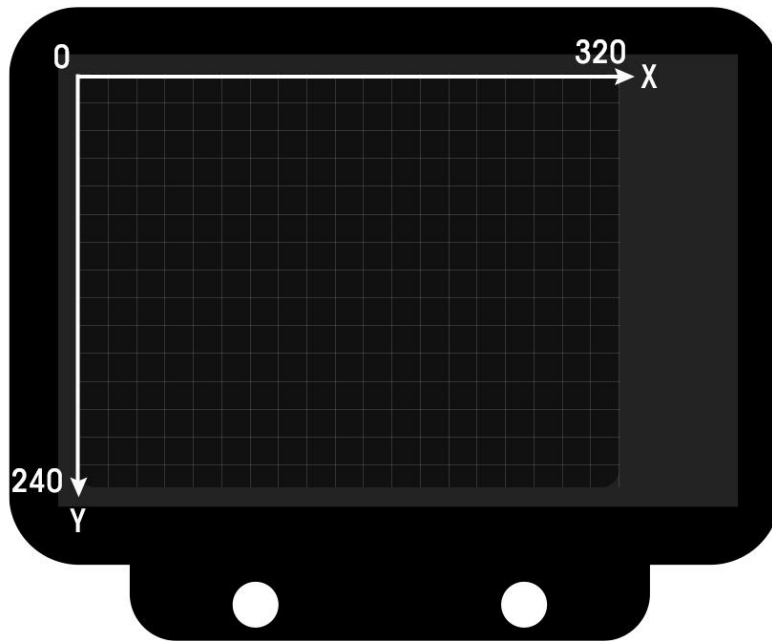
- 1. Image extraction:** first process the image and extract the required information.
- 2. Color space conversion:** Convert RGB color space to HSV or LAB color space to represent colors more efficiently.
- 3. Threshold processing:** Extract areas of specific colors through threshold segmentation algorithm.
- 4. Contour detection:** Use contour detection algorithm to find the boundary of color blocks.



II . Color Tracking Principle

1. The K210 Module Screen Coordinates

The screen coordinates of the K210 module are based on the upper left corner as the origin. The horizontal axis is the X-axis with a maximum value of 320, and the vertical axis is the Y-axis with a maximum value of 240. The K210 module will select the object to be tracked in real time. When the coordinates of the object relative to the screen change, it will return the new coordinate values in real time, and the AI car will execute the corresponding motion instructions according to the changes in coordinates.

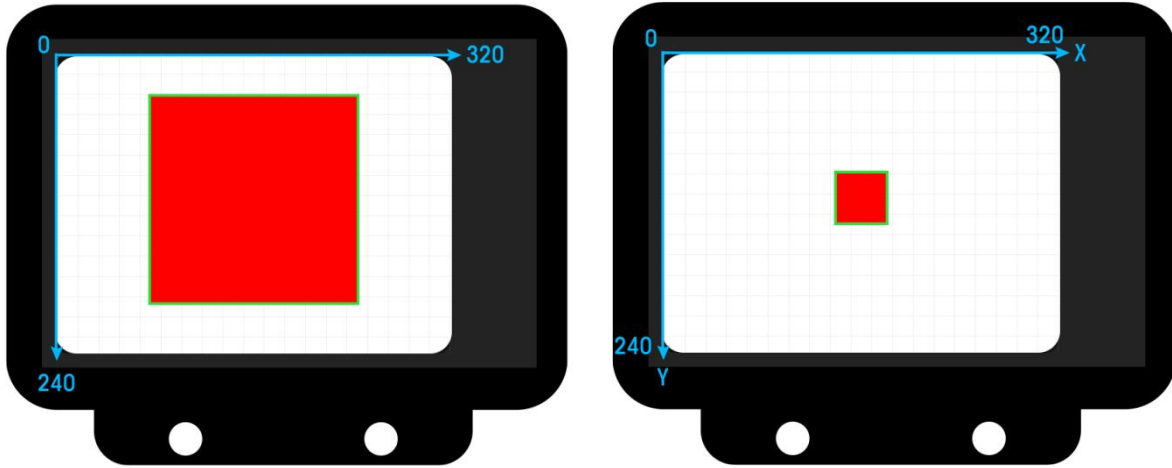


2. AI car color tracking logic

(1) The forward and backward movement of the AI car

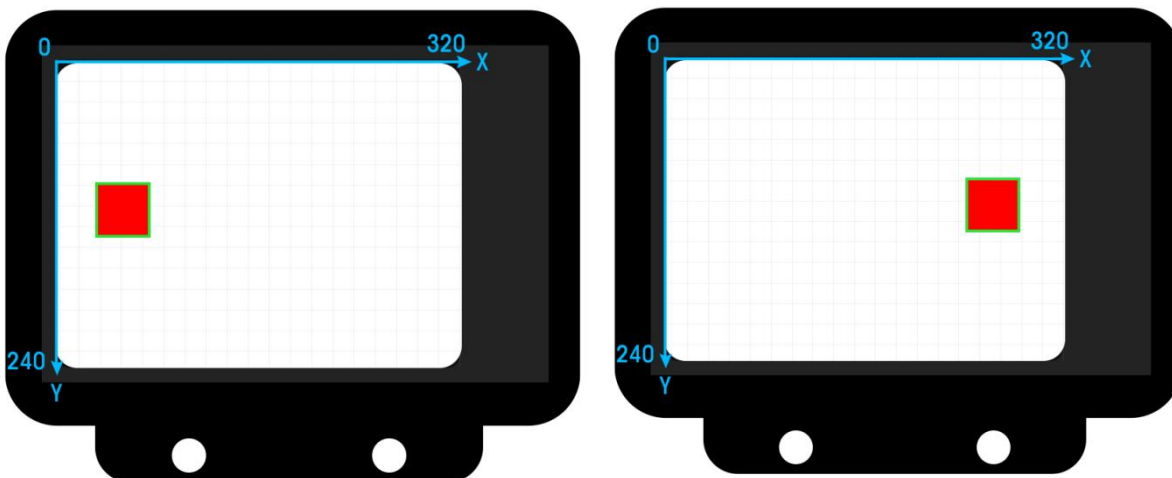
When the object is close to the K210 module, the height value of the green box in the screen of the object will increase. At this time, the program will control the AI car to move backward until the height of the box of the object remains the original size. When the object moves away from the K210 module, the height value of the green box of the object

within the screen will decrease. At this time, the program will control the AI car to move forward until the height of the box of the object remains at its original size.



(2) The AI car rotates left and right

When the object shifts to the left within the screen, the X-coordinate value of the green box of the object will decrease, and the program will control the AI car to rotate and move to the left until the box of the object remains at the center of the screen. When the object shifts to the right on the screen, the x-coordinate value of the green box of the object will increase, and the program will control the AI car to rotate and move to the right until the box of the object remains at the center of the screen.



III. Color Tracking Program

1. Sample Program

Open the "[3.Color_Tracking.py](#)" file located in "English\Python\3.Program". Connect the ESP32 controller board to the computer using a USB cable, select the correct controller board and port, upload the code to the ESP32 controller board, and then connect the K210 module to the ESP32 controller board.

The reference procedure is as follows:

```
from libs import ACB_Canmv
from libs import ACB_SmartCar_V2
import time
cam = ACB_Canmv.ACB_Canmv()
cam.init(cam.SDA, cam.SCL)
car = ACB_SmartCar_V2.ACB_SmartCar_V2()
car.Move(car.Stop, 0)

X_STOP_MIN = 30
X_STOP_MAX = 240
H_STOP_MIN = 50
H_STOP_MAX = 70
COLOR_INDEX = 3
PIXELS_TH = 200
AREA_TH = 200
last_seen_ms = time.ticks_ms()

while True:
    if cam.color_recognize(COLOR_INDEX, PIXELS_TH, AREA_TH):
        x = cam.getCX
        H = cam.getH
        print("cx =",x, ",H =",H)
        if (X_STOP_MIN <= x <= X_STOP_MAX) and (H_STOP_MIN <= H <=
H_STOP_MAX):
            car.Move(car.Stop, 0)
        if H > H_STOP_MAX:
            car.Move(car.Backward, 150)
        elif H < H_STOP_MIN:
            car.Move(car.Forward, 150)
        if x > X_STOP_MAX:
            car.Move(car.Clockwise, 150)
        elif x < X_STOP_MIN:
            car.Move(car.Contrarotate, 150)
        last_seen_ms = time.ticks_ms()
        if time.ticks_diff(time.ticks_ms(), last_seen_ms) > 200:
            car.Move(car.Stop, 0)
        time.sleep_ms(50)
```

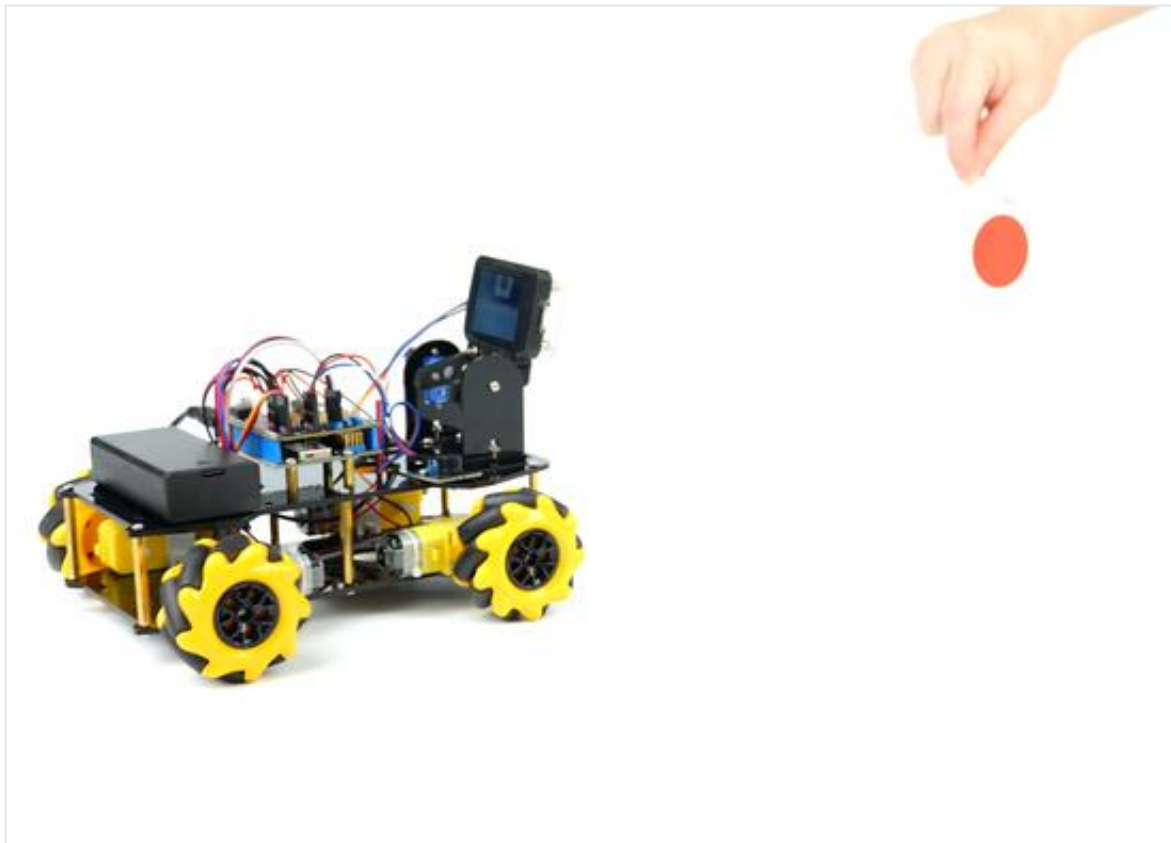
NOTE: Changing the number in the program will change the color the car will track.

“if cam.color_recognize(COLOR_INDEX, PIXELS_TH, AREA_TH):”

The value of COLOR_INDEX is 0 for all colors, 1 for red, 2 for green, and 3 for blue

2.Program Functions

When the program is uploaded, place the red card in front of the camera of the K210 module, and the screen will use a box to select the color of the card. When the card is away from the K210 module, the car will move forward, when the card is close to the K210 module, the car will move backward, when the card moves to the left, the car will rotate to the left, and when the card moves to the right, the car will rotate to the right.



Lesson 4 Face Recognition

I . The Principle of Face Recognition

Face recognition is a biometric technology that identifies people based on their facial features. It uses a camera or camcorder to collect images or video streams containing faces, automatically detects and tracks faces in images, and then performs facial recognition on the detected faces. This series of related technologies are also commonly called portrait recognition or facial recognition.



FaceIdentificationThe system mainly consists of six components: face image acquisition, face detection, face image preprocessing, face image feature extraction, face feature matching and face feature recognition.

1. Face image collection

First, you need to obtain the image or video to be recognized. This can be achieved through a camera, photo or video file.

2. Face Detection

Identify areas in an image that may be faces. This is usually done using computer vision techniques, such as feature-based methods, machine learning algorithms, or deep learning neural networks. Common algorithms include Haar cascade detectors, face detectors based on convolutional neural networks (CNNs), etc.

3. Face Image Preprocessing

Image preprocessing for faces is a process of processing images based on face detection results and ultimately serving feature extraction. Due to various restrictions and random interference, the original images obtained by the system cannot be used directly. The detected face area must be normalized in the early stage of image processing for subsequent feature extraction and matching. The preprocessing process mainly includes rotating the face to the standard position, adjusting the scale, cropping, light compensation of the face image, grayscale transformation, histogram equalization, geometric correction, filtering, and sharpening.

4. Face Image Feature Extraction

Face feature extraction is the process of modeling the features of a face. Information that can describe its features is extracted from the face image, and the appearance and structure of the face are converted into a vector or feature descriptor. Feature data that helps in face classification is obtained based on the shape description of the facial organs and the distance characteristics between them. Its feature components usually include the Euclidean distance, curvature and angle between feature points. The face is composed of parts such as eyes, nose, mouth, and chin. The geometric description of these parts and the structural relationship between them can be used as an important feature for identifying faces. These features are called geometric features.

Knowledge-based face representation mainly includes methods based on geometric features and template matching methods.

5. Facial Feature Matching

The feature data of the extracted face image is searched and matched with the feature template stored in the database. By setting a threshold, when the similarity exceeds this threshold, the matching result is output. Various similarity measurement methods can be used in this step, such as Euclidean distance, cosine similarity, and classifier based on Support Vector Machine (SVM).

6. Facial Feature Recognition

Face recognition is to compare the facial features to be identified with the facial feature templates that have been obtained, and judge the identity information of the face based on the degree of similarity. This process can be divided into two categories: one is confirmation, which is a one-to-one image comparison process, and the other is identification, which is a one-to-many image matching and comparison process.

II. Face Detection

1. Sample Program

Open the ["4.1Face_recognition.py"](#) file located in "English\Python\3.Program". Connect the ESP32 controller board to the computer using a USB cable, select the correct controller board and port, upload the code to the ESP32 controller board, and then connect the K210 module to the ESP32 controller board.

The reference procedure is as follows:


```
from libs import ACB_Canmv
import time
from machine import Pin, PWM

Yservo_PIN = 25
Camera_Angle = 90
cam = ACB_Canmv.ACB_Canmv()
cam.init(cam.SDA, cam.SCL)
pwm = PWM(Pin(Yservo_PIN), freq=50)

def servo_write(angle):
    us = 500 + int((angle / 180) * 2000)
    duty = int(us * 1024 // 20000)
    pwm.duty(duty)
servo_write(Camera_Angle)

while True:
    if cam.face_recognize():
        print("Tag:", cam.getTag)
        print("CX:", cam.getCX)
        print("CY:", cam.getCY)
        print("W:", cam.getW)
        print("H:", cam.getH)
        print()
    time.sleep_ms(50)
```

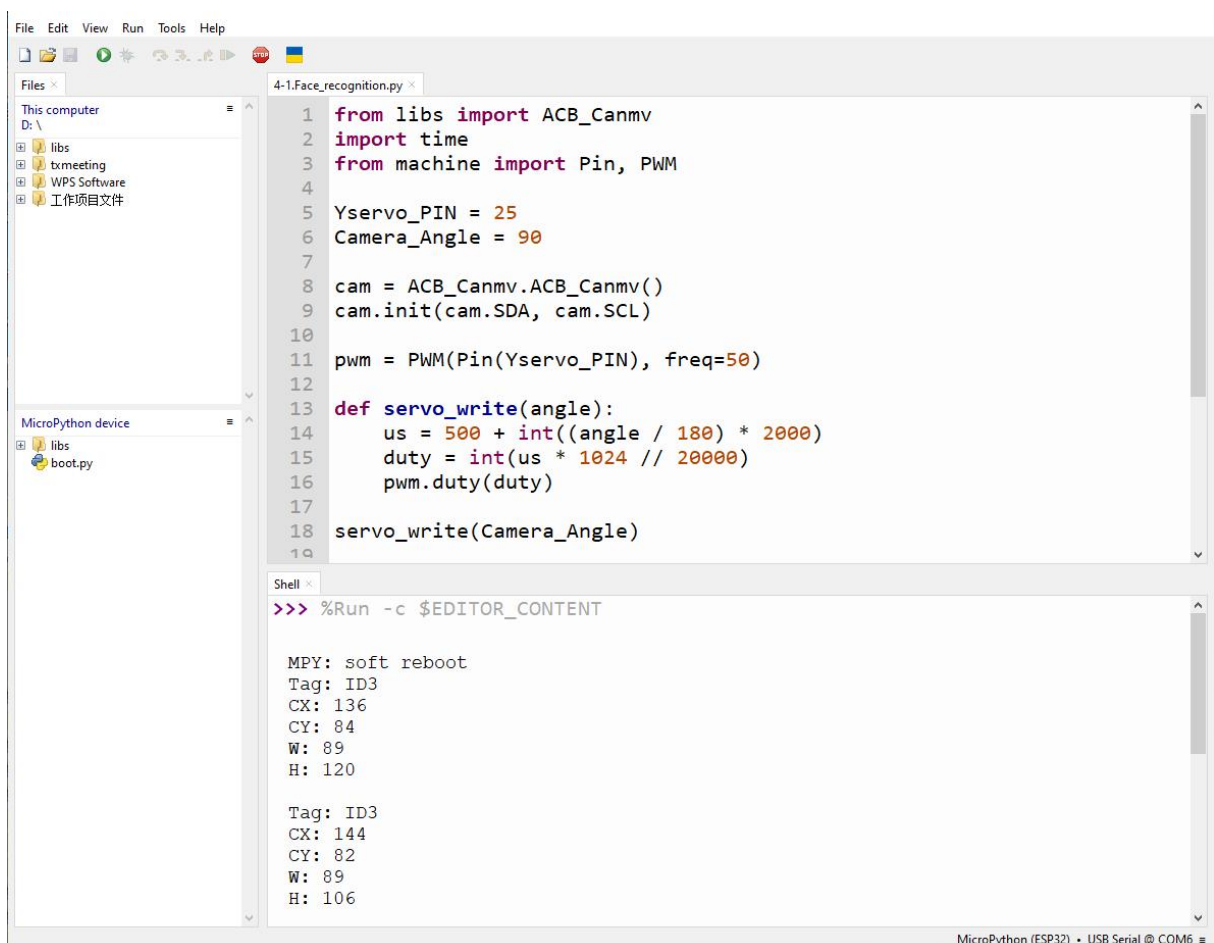
2.Program Functions

When the program is uploaded, place the card with the face in front of the camera of the K210 module, and the screen will use a box to select the face in the card. When you press the S2 button, the K210 module will record the current face and assign a face label. The next time the same face is recognized, the screen will automatically select and display the face label.



Note: Long press the S1 button to delete the saved face tags

Open the Thonny console, and it will print the label of the face, the position information of the face, and the values of the length and height of the box.



```

File Edit View Run Tools Help
Files
This computer
D:\
  libs
  txmeeting
  WPS Software
  工作项目文件

MicroPython device
  libs
  boot.py

4-1.Face_recognition.py
1 from libs import ACB_Canmv
2 import time
3 from machine import Pin, PWM
4
5 Yservo_PIN = 25
6 Camera_Angle = 90
7
8 cam = ACB_Canmv.ACB_Canmv()
9 cam.init(cam.SDA, cam.SCL)
10
11 pwm = PWM(Pin(Yservo_PIN), freq=50)
12
13 def servo_write(angle):
14     us = 500 + int((angle / 180) * 2000)
15     duty = int(us * 1024 // 20000)
16     pwm.duty(duty)
17
18 servo_write(Camera_Angle)
19

Shell
>>> %Run -c $EDITOR_CONTENT

MPY: soft reboot
Tag: ID3
CX: 136
CY: 84
W: 89
H: 120

Tag: ID3
CX: 144
CY: 82
W: 89
H: 106

MicroPython (ESP32) • USB Serial @ COM6
  
```

III. Facial Recognition Starts the Smart Car

1. Sample Program

Open the "[4.2Face_task.py](#)" file located in "English\Python\3.Program". Connect the ESP32 controller board to the computer using a USB cable, select the correct controller board and port, upload the code to the ESP32 controller board, and then connect the K210 module to the ESP32 controller board.

The reference procedure is as follows:

```
from libs import ACB_Canmv
from libs import ACB_SmartCar_V2
import time

Yservo_PIN = 25
Camera_Angle = 90
cam = ACB_Canmv.ACB_Canmv()
cam.init(cam.SDA, cam.SCL)
car = ACB_SmartCar_V2.ACB_SmartCar_V2()
car.Move(car.Stop, 0)
tag = "None"
lastTagChangeTime = time.ticks_ms()

def Car_task():
    global tag
    time.sleep_ms(1000)
    for _ in range(4):
        car.Move(car.Forward, 200)
        time.sleep_ms(1450)
        car.Move(car.Contrarotate, 200)
        time.sleep_ms(800)
    tag = "None"
    car.Move(car.Stop, 0)
```

```
while True:
    if cam.face_recognize():
        tag = cam.getTag
        print(tag)
        lastTagChangeTime = time.ticks_ms()
    if tag == "ID1" and time.ticks_diff(time.ticks_ms(), lastTagChangeTime) > 500:
        Car_task()
    time.sleep_ms(20)
```

2.Program Functions

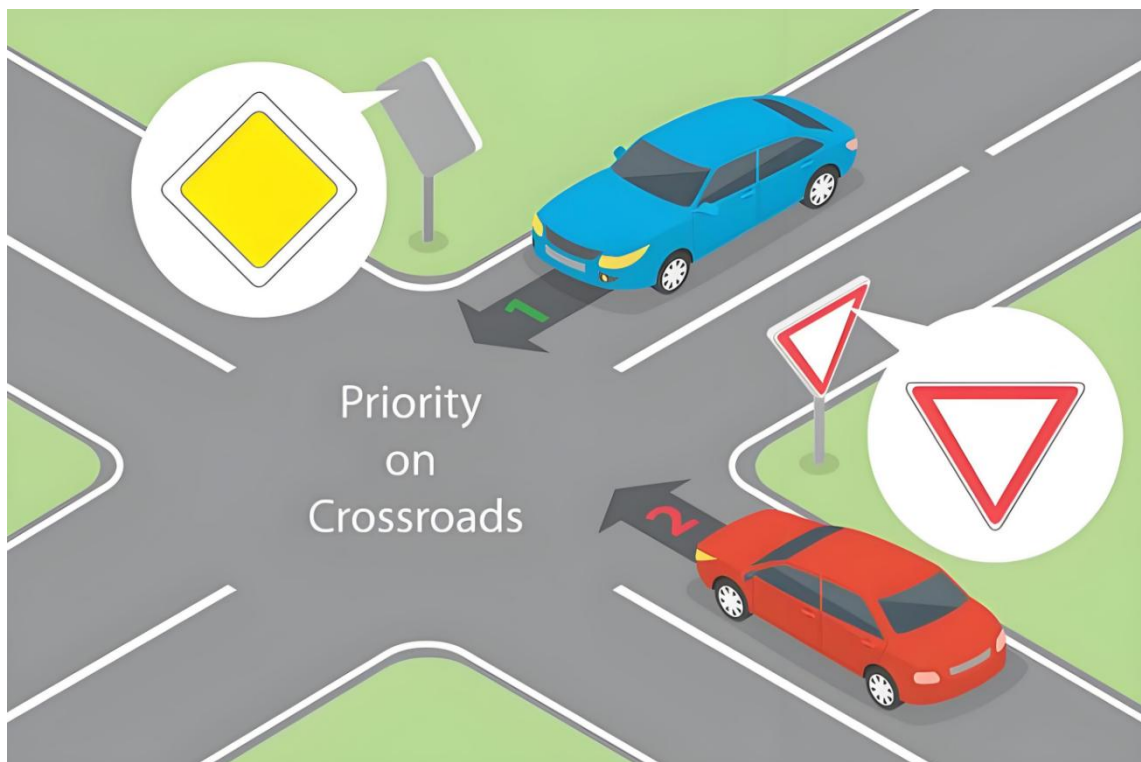
After the program is uploaded, place the card with the face in front of the camera of the K210 module. The screen will use a box to select the face in the card. When the label of the recognized face is "ID1", after removing the card, the car will start and follow a quadrilateral track.

Lesson 5 Road Sign Recognition

I . Principles of Road Sign Identification

Road sign recognition technology is a combination of computer vision and machine learning technology, and is mainly used in the fields of autonomous driving, driver assistance systems, and intelligent transportation. Road sign recognition relies on multiple technologies such as image acquisition, preprocessing, feature extraction, object detection, classification, and recognition, especially deep learning plays a very critical role in it. With the development of technology, the accuracy and real-time performance of road sign recognition systems are constantly improving, and it has gradually become an important part of autonomous driving and intelligent transportation systems.

The core technologies of road sign recognition mainly include traditional image processing methods based on SVM and HOG algorithms and deep learning methods based on CNN.



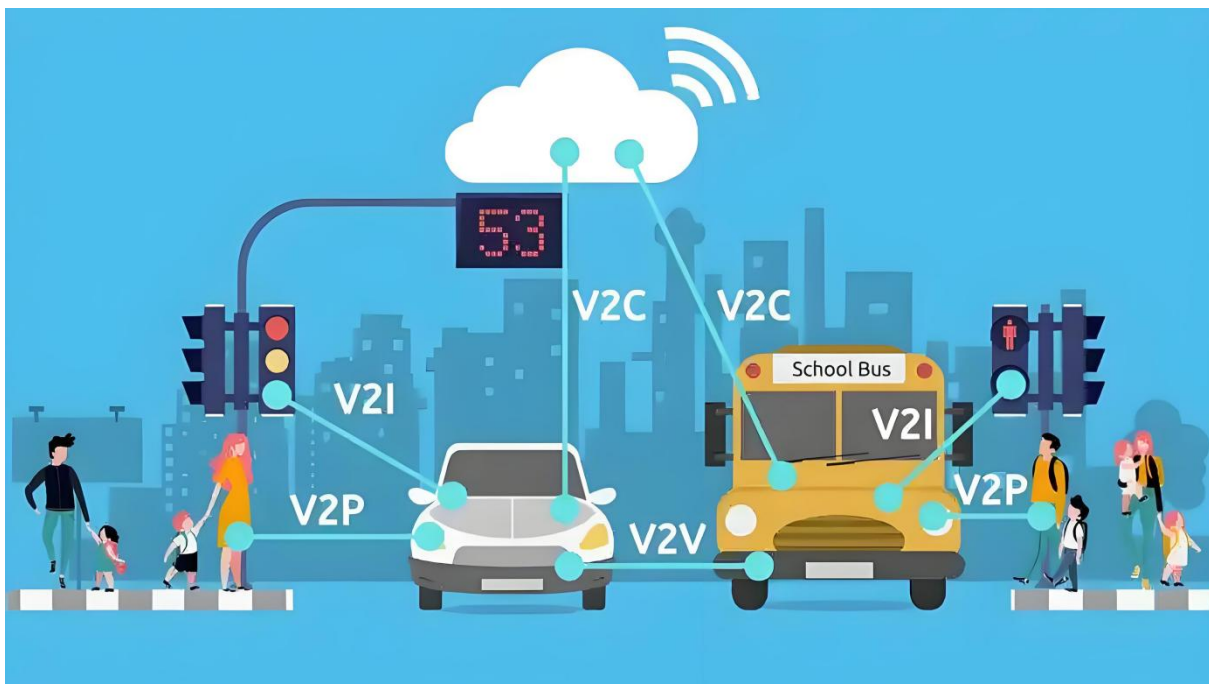
II . Application of Road Sign Recognition

1. Traffic Sign Recognition (TSR)

The TSR system uses a front-view camera to identify traffic signs on the road, such as no parking, no entry, construction, no U-turn, etc. The system can collect and identify traffic sign information, issue instructions or warnings to the driver, and even directly control the vehicle to ensure smooth traffic and prevent accidents. For example, the TSR system can identify speed limit signs and automatically adjust the vehicle speed according to the speed limit value to ensure that it does not exceed the speed limit.

2. Autopilot Assist

In autonomous driving technology, road sign recognition is one of the key technologies. By identifying various signs on the road, the autonomous driving system can make corresponding driving decisions, such as slowing down, avoiding, turning, etc. This greatly improves the safety and reliability of autonomous driving.



III. Procedure for Road Sign Identification

1. Sample Program

Open the ["5.1Traffic_sign.py"](#) file located in "English\Python\3.Program\5.1Traffic_sign". Connect the ESP32 controller board to the computer using a USB cable, select the correct controller board and port, upload the code to the ESP32 controller board, and then connect the K210 module to the ESP32 controller board.

The reference procedure is as follows:

```
from libs import ACB_Canmv
from libs import ACB_SmartCar_V2
import time

cam = ACB_Canmv.ACB_Canmv()
cam.init(cam.SDA, cam.SCL)
car = ACB_SmartCar_V2.ACB_SmartCar_V2()
car.Move(car.Stop, 0)
tag = "None"
lastTagChangeTime = time.ticks_ms()

def Car_task():
    global tag
    if tag == "Go_Straight":
        car.Move(car.Forward, 180)
        time.sleep_ms(500)

    elif tag == "Turn_Right":
        car.Move(car.Clockwise, 180)
        time.sleep_ms(500)

    elif tag == "Turn_Left":
        car.Move(car.Contrarotate, 180)
        time.sleep_ms(500)
```



```
elif tag == "Turn_Around":
    car.Move(car.Forward, 180)
    time.sleep_ms(1450)
    car.Move(car.Clockwise, 180)
    time.sleep_ms(1600)
    car.Move(car.Forward, 180)
    time.sleep_ms(1300)

elif tag == "Throughout":
    car.Move(car.Stop, 180)
    time.sleep_ms(500)

tag = "None"
car.Move(car.Stop, 180)

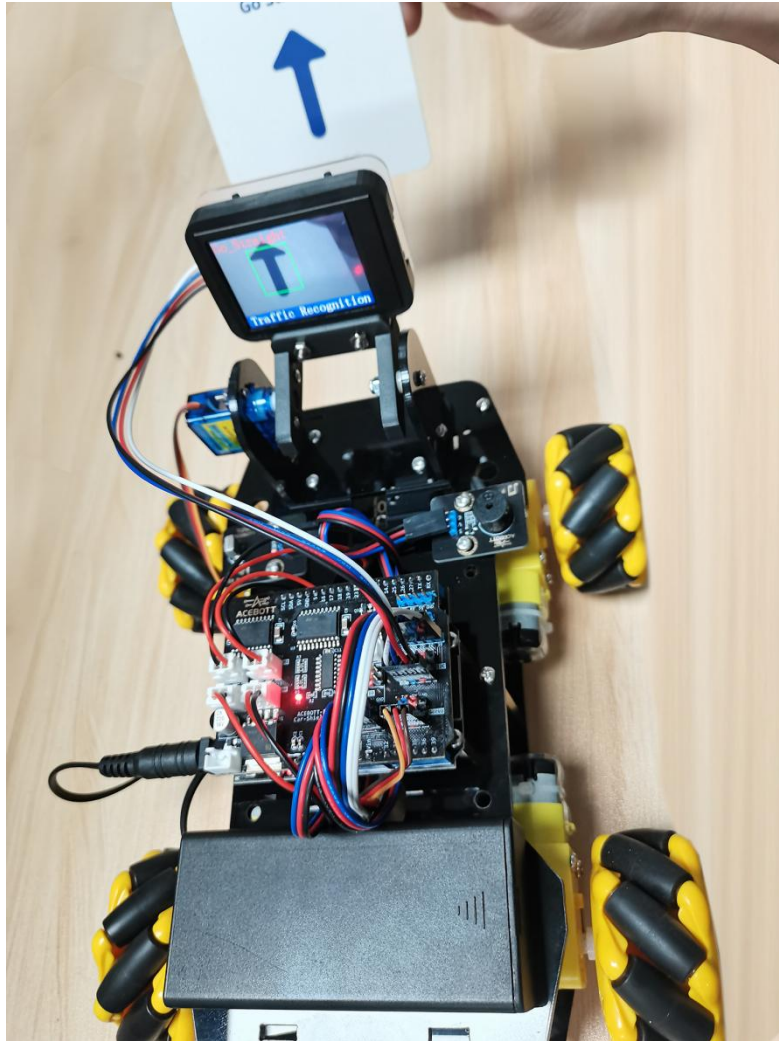
while True:
    if cam.Traffic_recognize(1):
        tag = cam.getTag
        print(tag)
        lastTagChangeTime = time.ticks_ms()

    if tag != "None" and time.ticks_diff(time.ticks_ms(), lastTagChangeTime) > 500:
        Car_task()

    time.sleep_ms(20)
```

2.Program Functions

After the program is uploaded, place the card with the traffic sign in front of the camera of the K210 module. The screen will use a box to select the traffic sign on the card. When the traffic sign card is removed, the car will complete the corresponding action according to the content of the label card.



IV. Line Inspection and Road Sign Recognition

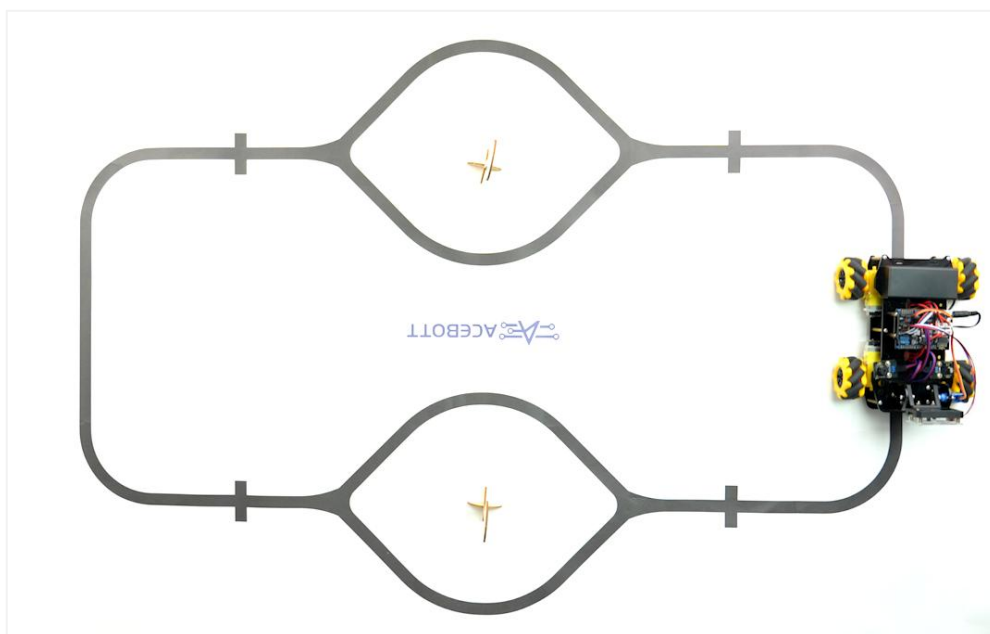
1. Sample Program

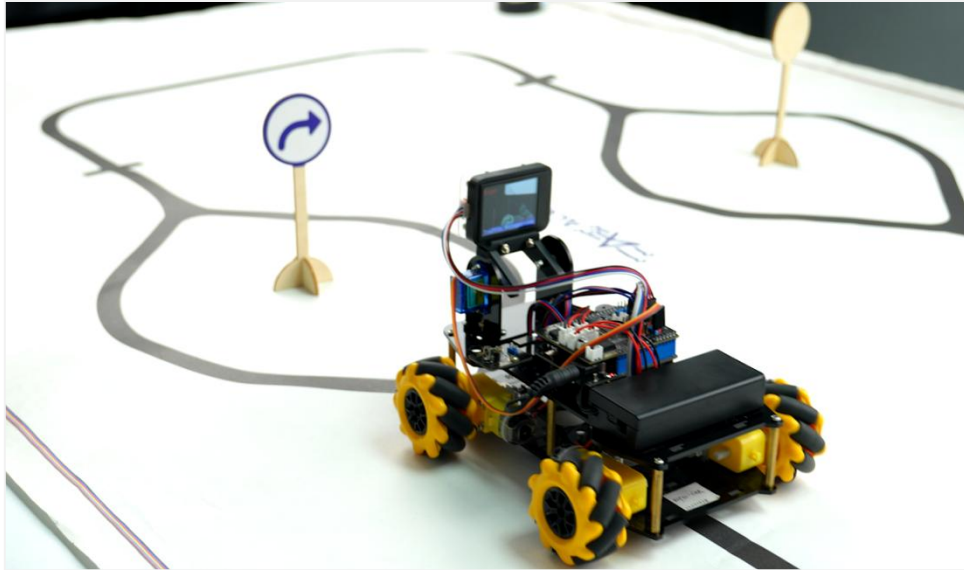
Open the ["5.2Traffic_task.py"](#) file located in "English\Python\3.Program\5.2Traffic_task". Connect the ESP32 controller board to the computer using a USB cable, select the correct controller board and port, upload the code to the ESP32 controller board, and then connect the K210 module to the ESP32 controller board.

2.Program Functions

After the program is uploaded, the car will complete the line inspection task on the line inspection map. At the fork in the road, the K210 module will recognize the road sign pattern ahead. According to the recognized direction, it will choose to move forward to the left or right. At the same time, it will also recognize the horn and speed limit signs on the road signs and perform corresponding functions.

Note: The location of the Patrol line map file is in the QD003 tutorial \ English\Python\6.Patrol Line Map \ [Road Sign Patrol Line Map \(100 x140cm\).pdf](#) of the tutorial. You need to print it by yourself according to the size inside.





Note: You can use the matching black tape to stick out the walking route for testing.

If the trolley fails to move normally during the line inspection process, it may be due to the following reasons:

1.The car is too slow.

To solve the problem, you can increase the value of the speed in the program. In the program, the speed range of the motor is 0 to 255, so you can appropriately increase the value, for example, 180, to ensure that the car has sufficient speed to move.

```
Speed = 120
```

```
car.Move(car.Forward, Speed)
```

2. Due to the interference of ambient light factors, ESP32 may mistakenly recognize white areas as black lines.

To solve this problem, you can first test the specific values of the patrol sensors on the white area and the black line, and accordingly adjust the value of the Black_Line variable in the program to make it closer to the actual measured value of the black line.

```
BLACK_LINE = 3000
```

Lesson 6 Visual Line Patrol

I . The Principle of Visual Line Patrol

Visual line patrol is a method that uses cameras, image processing algorithms and other technologies to identify, track and locate lines. The principle is to process line images, extract line feature information such as color, shape, texture, etc., and then patrol the line based on this information.

The visual line inspection system has the advantages of being fast, accurate, and flexible. It can automatically inspect the line, reduce the cost and risk of manual inspection, and improve the reliability and safety of the inspection. Visual line inspection technology has been widely used in industries such as electricity, railways, and petrochemicals, and has become one of the main means of modern line inspection.

II . Visual Line Patrol Algorithm

The visual line patrol algorithm is an algorithm that uses a camera to obtain track images and uses image processing technology to identify track lines, thereby controlling the robot to drive along a predetermined route. The algorithm mainly includes the following steps:

1. Image acquisition and preprocessing: First, the track image is acquired through the camera, and then the color image is converted into a grayscale image for subsequent processing. Grayscale conversion is usually performed using the weighted average method.
2. Image Binarization: Convert the grayscale image to a binary image by setting a threshold. Pixel values greater than the threshold are set to

white, and pixels less than the threshold are set to black. Common methods include Otsu's method.

3. Track line extraction: Extract the left and right edges of the track from the binary image using methods such as the eight-neighborhood method or the maze method. Then find the longest white column and calculate its average value as the center line.

4. Perspective transformation: Since the images captured by the camera may be distorted, a perspective transformation is required to transform the area in front of the car body into a top-down view in order to more accurately identify the track lines.

5. Smart CAR control: Adjust the speed of the smart car motor according to the recognition results and control the steering of the robot.

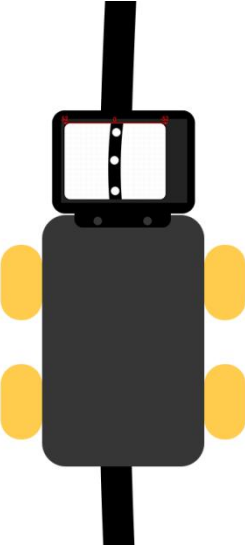
The application scenarios of the visual line patrol algorithm mainly include autonomous driving cars, robot competitions and other fields. Through precise image processing and control systems, the visual line patrol algorithm can enable the robot to accurately drive along the predetermined route, which has high practical value and research significance.



III. Intelligent Car Visual Line Patrol Principle

During the visual line inspection process, when black lines appear on the screen, the program will draw positioning points at the top, middle and bottom of the lines. The weights of these three positioning points decrease successively from top to bottom. After these three positioning points complete the visual line inspection algorithm, the numerical parameters of the current line inspection position relative to the car will be returned on the screen, with a range of -52 to 52.

The specific numerical range and the relationship between the movement of the trolley are as follows in the table:

State Diagram	State Description	Execution Status
	When the values returned by the three points of the line inspection on the screen range from -10 to 15, it indicates that the trolley is in the middle of the line inspection.	The car moves forward.
	When the values returned by the three points of the line inspection on the screen range from -52 to -10, it indicates that the posture of the car is offset to the left relative to the line inspection direction.	The car rotates to the right
	When the values returned by the three points of the line inspection on the screen range from 15 to 52, it indicates that the posture of the car is offset to the right relative to the line inspection direction.	The car rotates to the left

IV. Intelligent Car Visual Line Patrol Program

1. Sample Program

Open the "[6.Visual_patrol.py](#)" file located in "English\Python\3.Program". Connect the ESP32 controller board to the computer using a USB cable, select the correct controller board and port, upload the code to the ESP32 controller board, and then connect the K210 module to the ESP32 controller board.

The reference procedure is as follows:

```
from libs import ACB_Canmv
from libs import ACB_SmartCar_V2
import time

Camera_Angle = 25
cam = ACB_Canmv.ACB_Canmv()
cam.init(cam.SDA, cam.SCL)

car = ACB_SmartCar_V2.ACB_SmartCar_V2()
car.Move(car.Stop, 0)
cam.servo_write(Camera_Angle)

while True:
    if cam.visual_patrol():
        visualData = cam.Visual_data
        if visualData < -10 and visualData >= -50:
            car.Move(car.Stop, 0)
            time.sleep_ms(30)
            car.Move(car.Clockwise, 90)
        elif visualData <= 15 and visualData >= -10:
            car.Move(car.Forward, 110)
        elif visualData <= 52 and visualData > 15:
            car.Move(car.Stop, 0)
            time.sleep_ms(30)
            car.Move(car.Contrarotate, 90)
        else:
            car.Move(car.Clockwise, 100)
            time.sleep_ms(10)

    time.sleep_ms(10)
```

Visual line inspection data description:

The return value range of the visual line inspection of the car is -52 to 52, and it is displayed in real time in the upper left corner of the display screen. Taking the exact center of the display screen as the coordinate origin, the

left side represents positive numbers and the right side represents negative numbers. Moreover, the further to the left, the larger the value; the further to the right, the smaller the value. If adjustment is needed, the range of the car's position deviating to the left and right can be modified according to this principle.



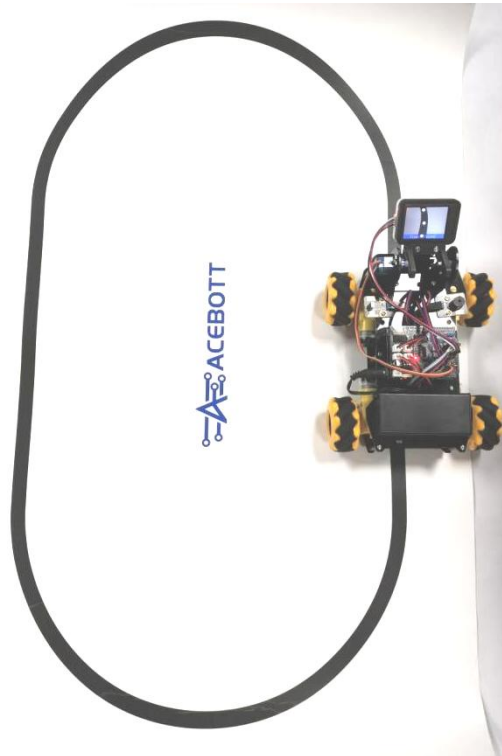
2.Program Functions

When the program is uploaded, place the QD003 smart car on the patrol map or use black tape to stick a patrol path on white paper. The screen will display a logo to identify the patrol line, and the car will move along the patrol path.

Note:

1. The location of the visual Patrol line map file is in the QD003 tutorial \ English\Python\6.Patrol Line Map \ [Visual Patrol Line Map \(80x120cm\).pdf](#) of the tutorial. You need to print it by yourself according to the size inside.
2. If you need to stick the line inspection path yourself, you should stick the path on white paper. The recommended size of the white paper is

80*120cm. To prevent the K210 module from mistakenly identifying the dark position on the outside, the size of the path should be less than 60*80cm, and the turning arc should be as gentle as possible, not a 90-degree right Angle.



Lesson 7 Machine Learning

I . Machine Learning

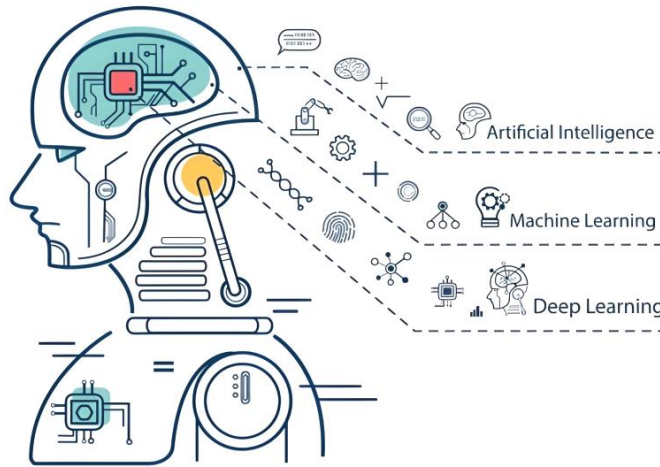
Machine learning is a branch of artificial intelligence that aims to enable computers to acquire knowledge and experience from data through learning and automated reasoning of computer systems, and use this knowledge and experience for pattern recognition, prediction and decision-making.

The core idea of machine learning is to use data to train computer algorithms so that they can automatically learn from the data and improve their performance without being explicitly programmed. By analyzing and interpreting large amounts of input data, machine learning algorithms can identify patterns and trends in the data and generate predictive models that can be applied to new data.

Machine learning algorithms can be divided into different types such as supervised learning, unsupervised learning and reinforcement learning. Supervised learning uses labeled training data to train the model to predict the label or target value of new data. Unsupervised learning is to discover hidden structures and patterns from data without labels. Reinforcement learning is to learn through interaction with the environment to maximize the cumulative reward.

Machine learning has a wide range of applications in various fields, such as image recognition, speech recognition, natural language processing, recommendation systems, financial forecasting, etc. It has become an important field in the development of modern science and technology and artificial intelligence, and has provided effective methods and tools for

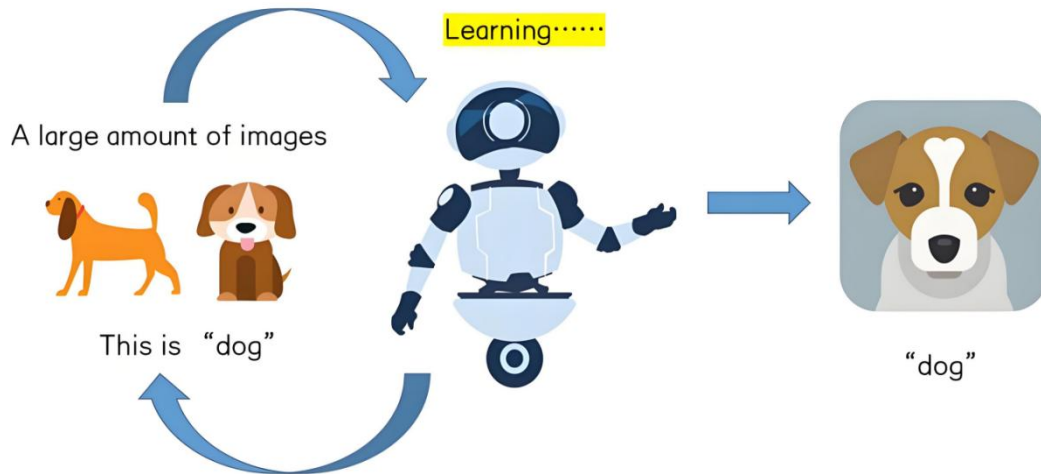
solving many practical problems.



II . Image Classification

Image classification is an image processing method that distinguishes different categories of objects according to their different features reflected in the image information. Image classification plays a key role in many applications, such as face recognition, object recognition, and autonomous driving.

The basic principle of image classification is to extract features from images and compare these features with pre-trained models to determine the category to which the image belongs. Common feature extraction methods include traditional hand-designed features and deep learning methods. Traditional hand-designed features usually include color features, texture features, and shape features, but these methods often do not work well when processing complex images. Deep learning methods can automatically learn more discriminative features from images by constructing deep neural networks.



III. Built-in Image Classification Application

1. Sample Program

Open the ["7.1Image_recognition.py"](#) file located in "English\Python\3.Program". Connect the ESP32 controller board to the computer using a USB cable, select the correct controller board and port, upload the code to the ESP32 controller board, and then connect the K210 module to the ESP32 controller board.

The reference procedure is as follows:

```
from libs import ACB_Canmv
import time

cam = ACB_Canmv.ACB_Canmv()
cam.init(cam.SDA, cam.SCL)

while True:
    if cam.image_recognize():
        print(cam.getTag)
        time.sleep_ms(50)
```

2.Program Functions

After the program is uploaded, place the "Bicycle" card, "Motorcycle" card, "Cat card", and "Water bottle" card in the card group respectively in front of the camera of the K210 module. The display screen of the K210 module will frame the currently recognized object and show the corresponding label.



IV. Machine learning Image Classification

1. Sample Program

Open the ["7.2Machine_learning.py"](#) file located in "English\Python\3.Program\7.2Machine_learning". Connect the ESP32 controller board to the computer using a USB cable, select the correct controller board and port, upload the code to the ESP32 controller board, and then connect the K210 module to the ESP32 controller board.

The reference procedure is as follows:

```
from libs import ACB_Canmv
import time

cam = ACB_Canmv.ACB_Canmv()
cam.init(cam.SDA, cam.SCL)

cam.machine_learning()

while True:
    time.sleep_ms(1000)
```

2.Program Functions

After the program is uploaded, place the card or object to be learned by the K210 module in front of its camera. When the card or object is clearly displayed on the screen, press the S2 button to record the screen information of the first angle, then change to the second angle, and press the S2 button to record. A total of 4 angles of screen information are recorded. The K210 module will learn the information of the current card or object and automatically generate a label.

Then change to another card or object and follow the same steps. We recommend learning up to three different objects. Finally, place the learned card or object in front of the camera of the K210 module, and its screen will display the corresponding label of the card or object.



Lesson 8 APP Control

I . APP Download

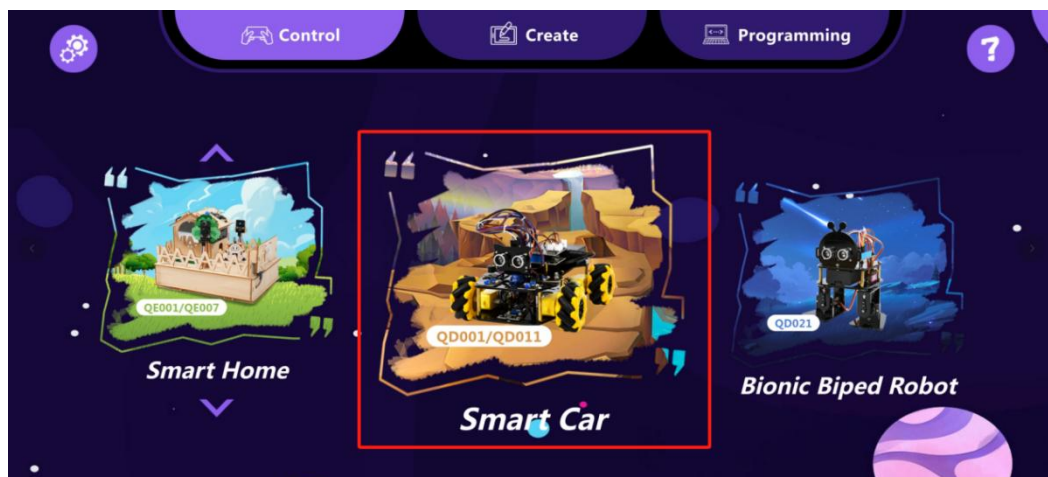
1. If you are using an iOS phone, you need to search for the keyword: ACEBOTT in the APP Store and then download it; if you are using an Android phone, you need to search for the keyword: ACEBOTT in the Google Play Store and then download it; the icon is shown in the figure below.



2. Open the app and enter the opening screen.



3. Enter the selection interface and select the image transmission vehicle.

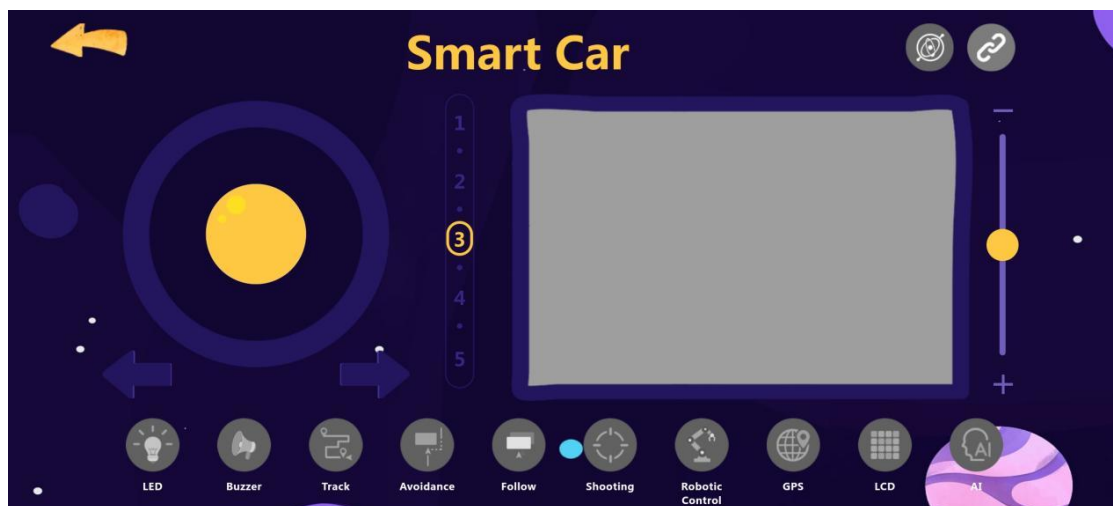


After clicking to enter, select "Control" to enter the car control page.



Note: You can click the Build button on the right to view the building video of this work.

4. Enter the image transmission car control interface (it cannot be controlled directly now, and the program needs to be uploaded).



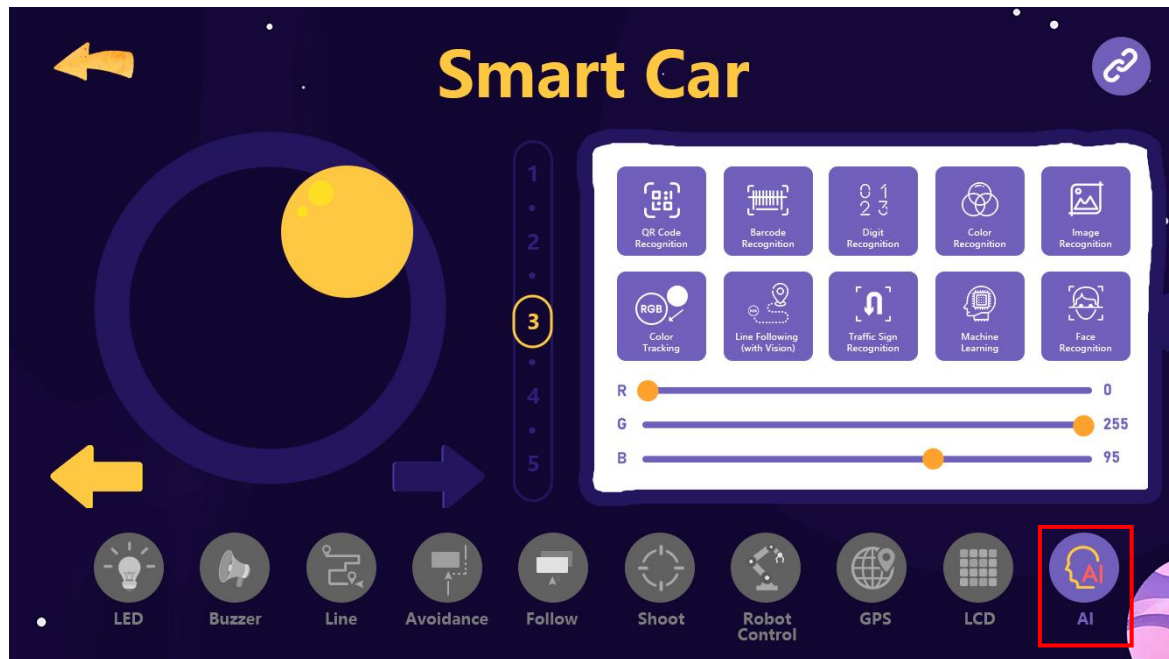
II . Program upload

Open the "[8.APP_Control.py](#)" file located in "English\Python\3.Program". Connect the ESP32 controller board to the computer using a USB cable, select the correct controller board and port, upload the code to the ESP32 controller board, and then connect the K210 module to the ESP32 controller board.

III. APP control instructions

1. AI function button

Click the AI function button at the bottom of the APP page to expand the AI control function page.



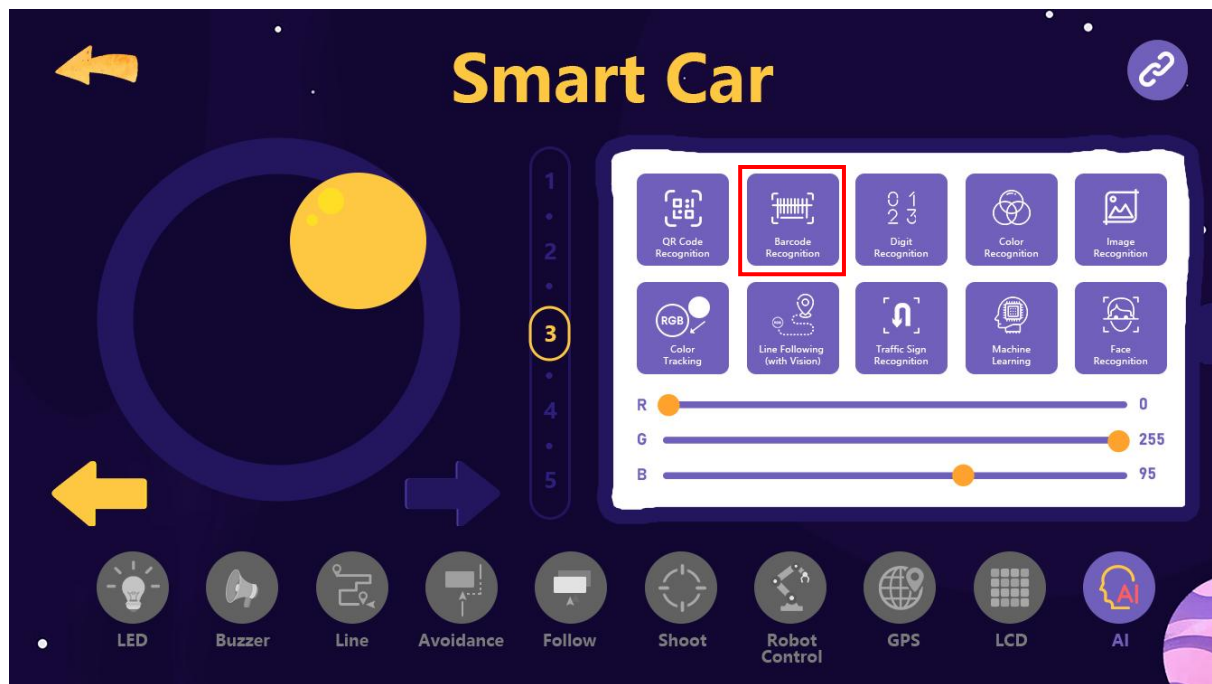
2. QR code recognition

After clicking the QR code recognition button, place the card with the QR code in front of the camera of the K210 module, and the screen will recognize the URL of the QR code.



3. Barcode recognition

After clicking the barcode recognition button, place the card with the barcode in front of the camera of the K210 module, and the screen will recognize the barcode information.



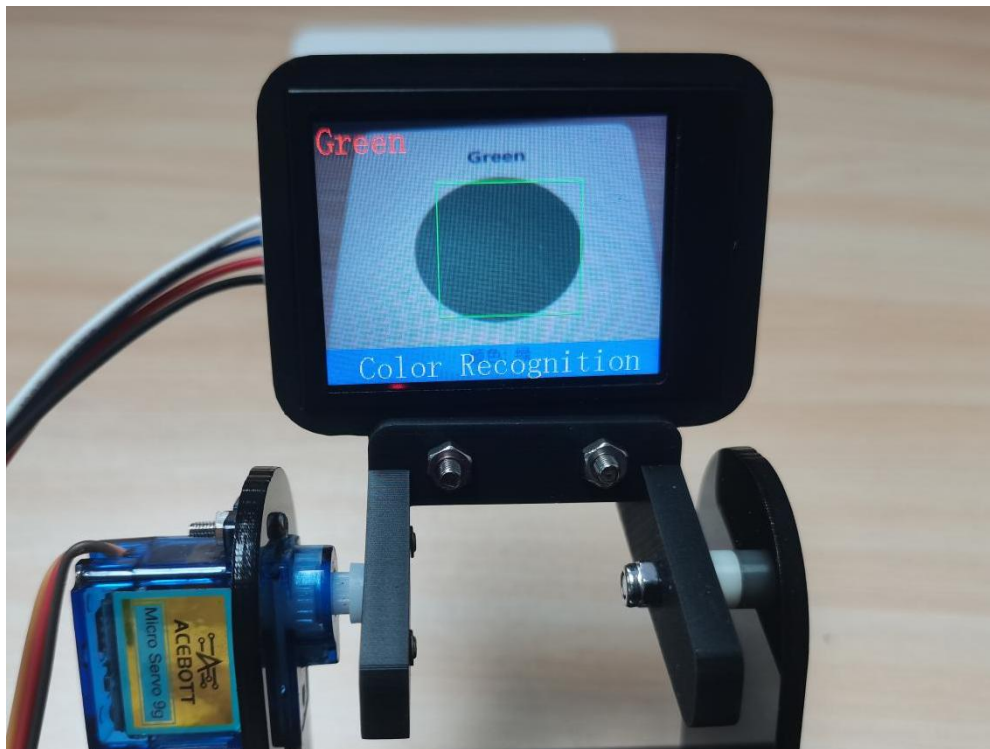
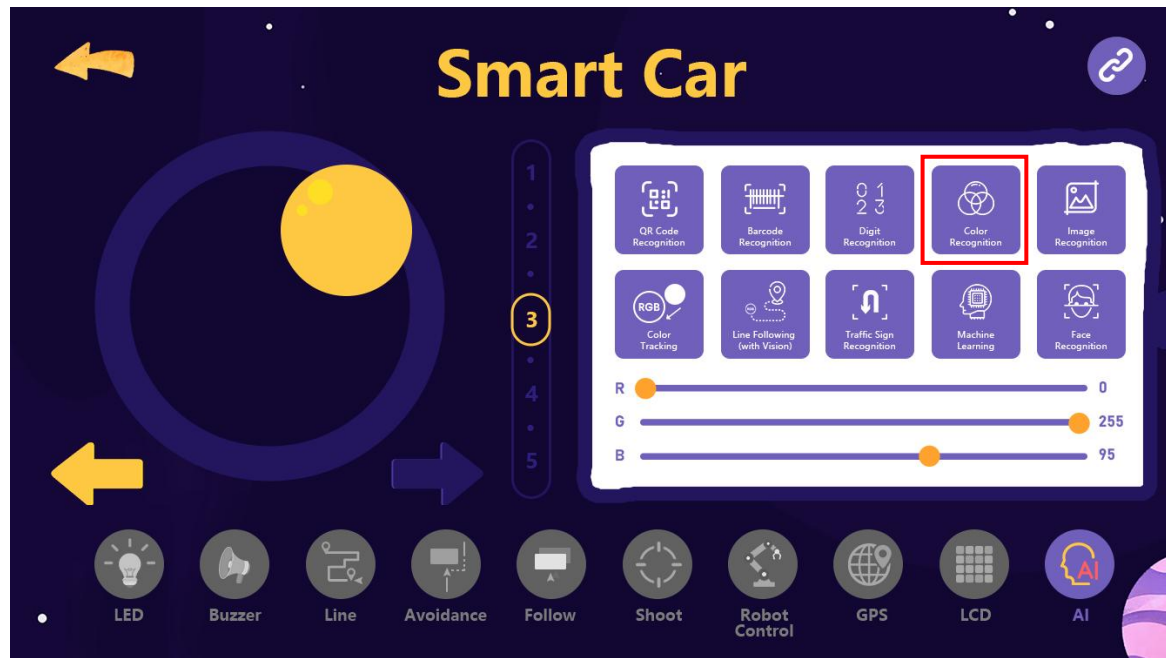
4. Digital Recognition

After clicking the digital recognition button, place the card with numbers in front of the camera of the K210 module, and the screen will recognize the digital information in the card.



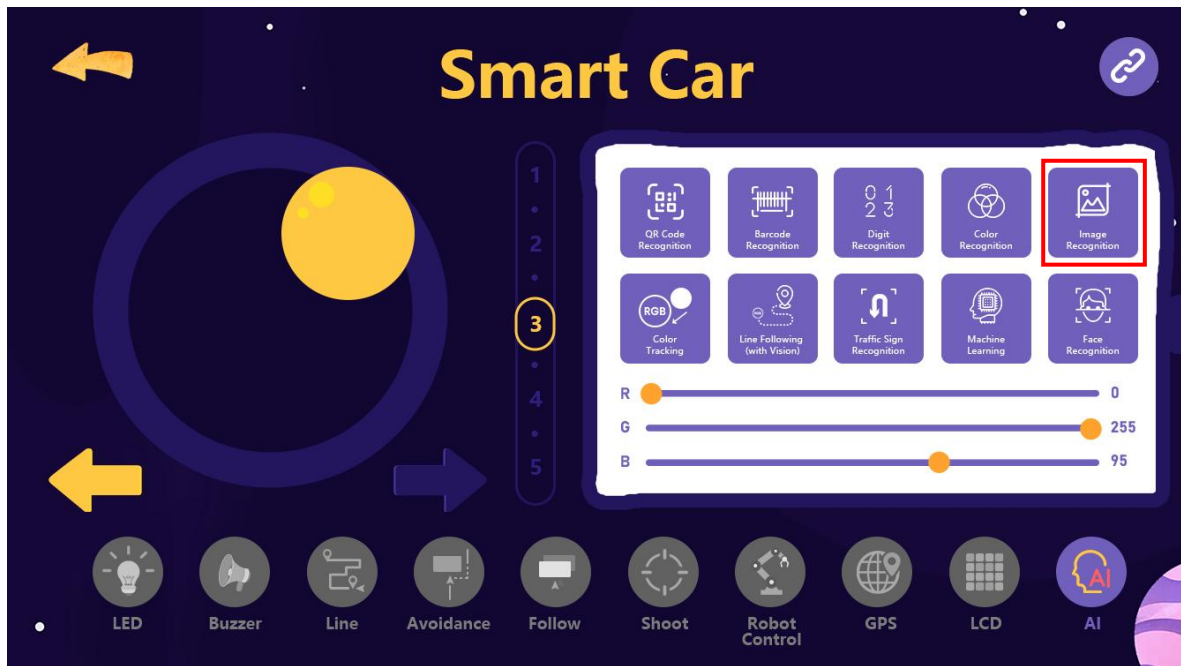
5. Color recognition

After clicking the color recognition button, place the colored card in front of the camera of the K210 module, and the screen will recognize the color information in the card.



6. Image Recognition

After clicking the image recognition button, place cards with images of different objects in front of the camera of the K210 module, and the screen will recognize the digital information in the card.



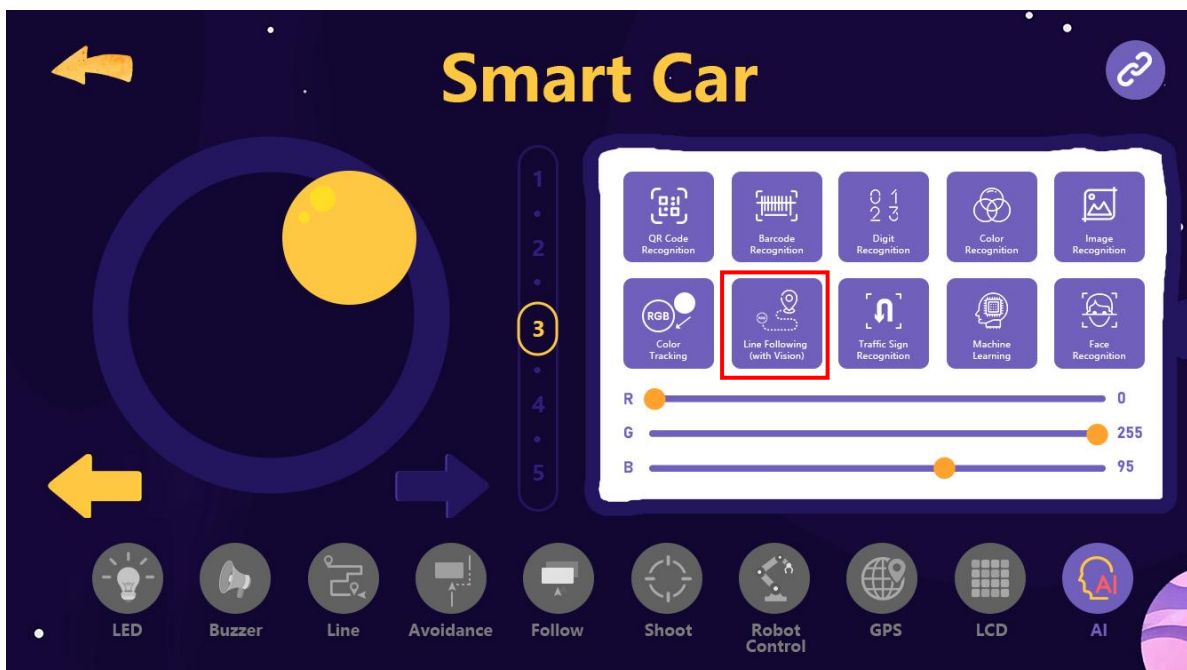
7. Color Tracking

After clicking the color tracking button, select a color and place the card with the selected color in front of the camera of the K210 module. The screen will recognize the digital information in the card and the smart car will move according to the color of the card.



8. Visual line inspection

After clicking the visual line patrol button, place the QD003 car on the line patrol map, and the car will move along the black line.



9. Road sign recognition

After clicking the road sign recognition button, place the card with the road

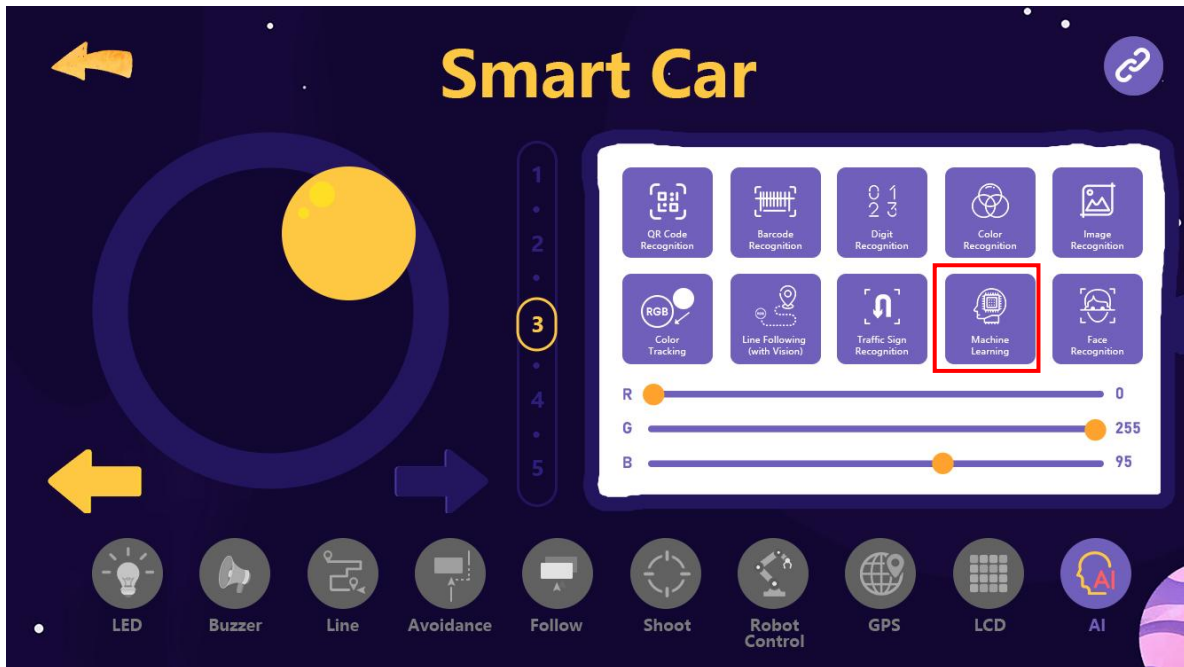
sign in front of the camera of the K210 module. The screen will recognize the road sign information in the card, and the smart car will move along with the road sign on the card.



10. Machine Learning

After clicking the machine learning button, place cards with different object images in front of the camera of the K210 module. The K210

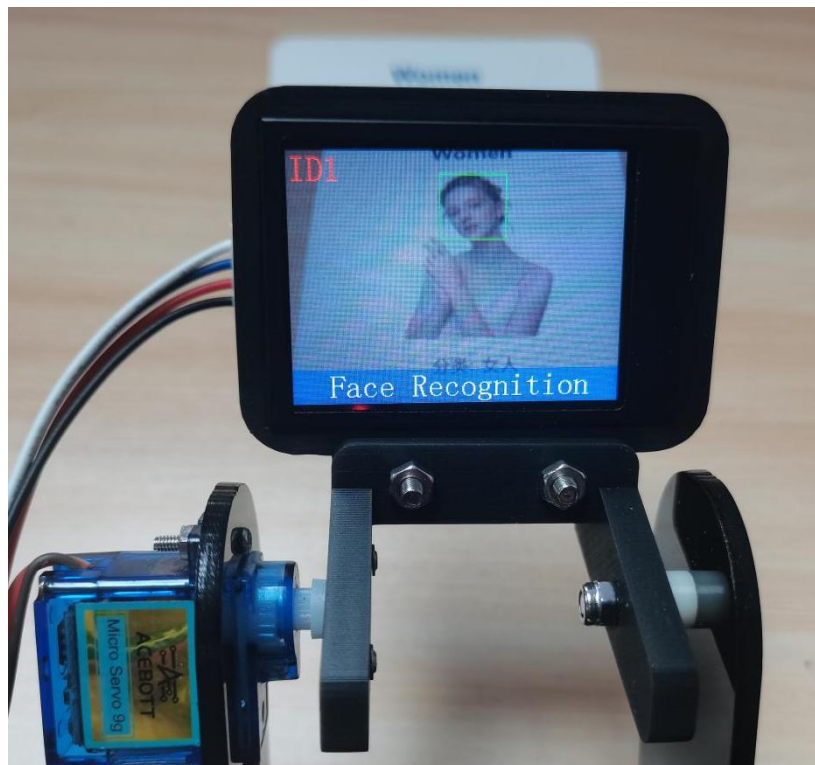
module will learn the information in the card. When the recognition is completed, when the recognized card is placed in front of the camera again, the K210 module will display its information.



11. Face Recognition

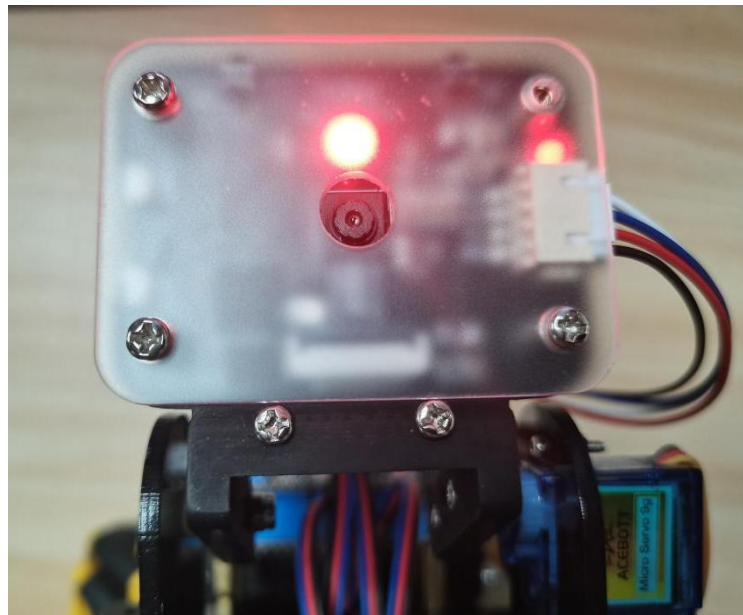
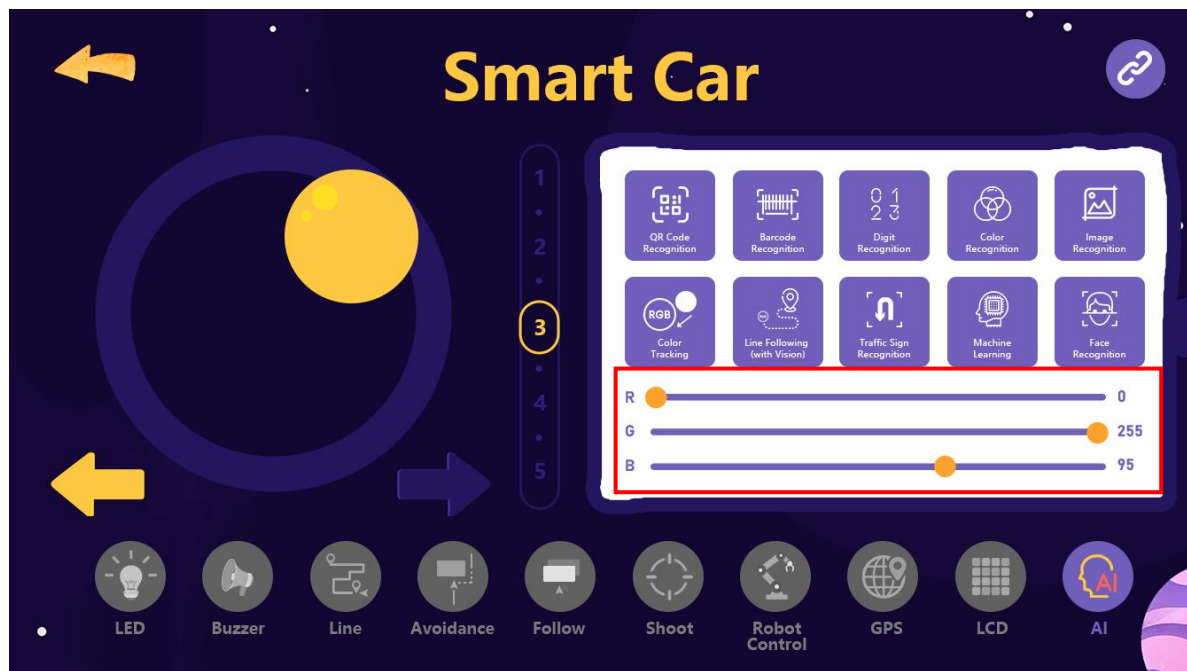
After clicking the face recognition button, place cards with different faces in front of the camera of the K210 module. The K210 module will learn the information in the card. When the recognition is completed, when the recognized card is placed in front of the camera again, the K210 module

will display its information.



12.RGB light adjustment

By sliding the R, G, and B sliders, you can control the color change of the RGB light on the K210 module. The value range is: 0~255.



Follow Us

Scan the QR codes to Follow Us for troubleshooting & the latest news.

We have a very large community that is very helpful for troubleshooting and we also have a support team at the ready to answer any questions.



ACEBOTT FB Group QR Code



YouTube QR Code